

VIETNAM NATIONAL UNIVERSITY - HO CHI MINH CITY
UNIVERSITY OF SCIENCE
FACULTY OF INFORMATION TECHNOLOGY



YOLO-based Object Detection Application

Statistical Machine Learning

21CNTThuc

INSTRUCTORS

NGO MINH NHUT
LE LONG QUOC

STUDENT

NGUYEN HONG HANH 21127503

Contents

I You only look once (YOLO)	2
1 Introduction	2
2 Architecture	2
II Applications	5
1 YOLO-based Object Detection Application	5
2 Vietnamese Food Detection Application	5
References	9

I You only look once (YOLO)

1 Introduction

You Only Look Once (YOLO), first introduced by Joseph Redmon and colleagues in 2016, is a real-time object detection system that applies a single convolutional network to the full image, dividing it into grid cells, and predicting bounding boxes and probabilities for each grid cell simultaneously.

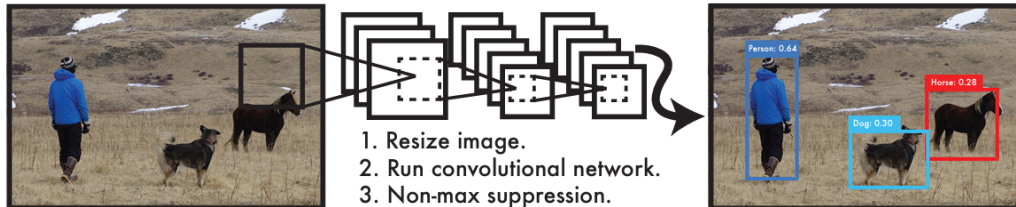


Figure 1: The YOLO Detection System

There are three key features:

1. **YOLO is extremely fast:** As YOLO approaches detection as a regression problem, there's no need for a complex pipeline. Instead, it directly applies the neural network to new images during testing to predict detections, enabling real-time processing of streaming video with minimal latency.
2. **YOLO reasons globally about the image when making predictions:** Unlike sliding window and region proposal-based techniques, YOLO sees the entire image during training and test time so it implicitly encodes contextual information about classes as well as their appearance.
3. **YOLO learns generalizable representations of objects:** Since YOLO is highly generalizable it is less likely to break down when applied to new domains or unexpected inputs.

YOLO has undergone significant development since its inception, evolving through several versions to improve its performance and capabilities. This project will use YOLOv6, the sixth version of YOLO, to train a new model for detecting new objects.

2 Architecture

The input image is first divided into $S \times S$ grid. For each grid cell, the model predicts B bounding boxes, confidence scores for those boxes, and C class probabilities. The output then will be a tensor of size $S \times S \times (5 \times B + C)$.

Confidence scores reflect how confident the model is that the box contains an object and also how accurate it thinks the box is that it predicts, defined as:

$$Pr(object) * IoU_{pred}^{truth},$$

where IoU_{pred}^{truth} is the intersection over union (IOU) between the predicted box and the ground truth.

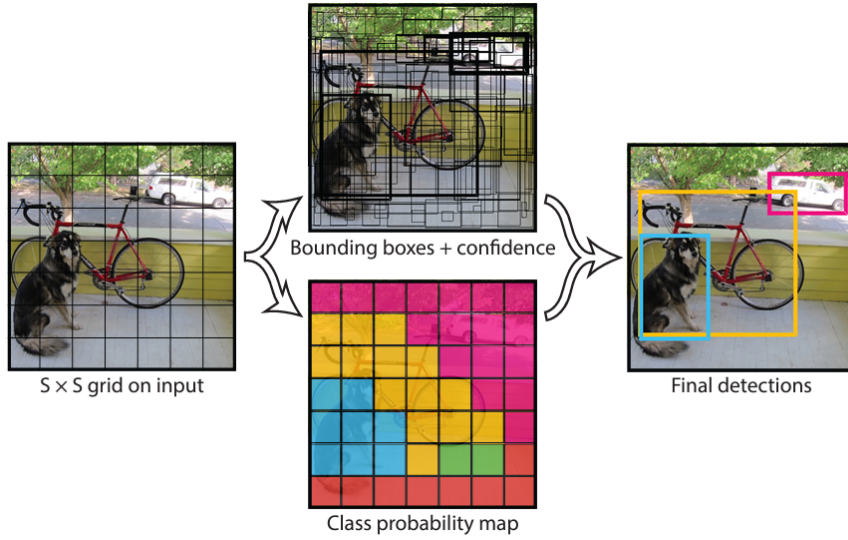


Figure 2: The Model

Each bounding box consists of 5 predictions: x, y, w, h and confidence. The (x, y) coordinates represent the center of the box relative to the bounds of the grid cell while (w, h) represent the width and height of the bounding box.

Each grid cell also predicts C conditional class probabilities $Pr(Class_i|Object)$. Only one set of class probabilities is predicted per grid cell, regardless of the number of boxes B .

At test time, class-specific confidence scores for each box are given by the multiplication of the conditional class probabilities and the individual box confidence predictions

$$Pr(Class_i|Object) * Pr(object) * IoU_{pred}^{truth} = Pr(Class_i) * IoU_{pred}^{truth}.$$

The network architecture is inspired by the GoogLeNet model for image classification. The network consists of 24 convolutional layers followed by 2 fully connected layers. Instead of using the inception modules found in GoogLeNet, it utilizes 1×1 reduction layers followed by 3×3 convolutional layers.

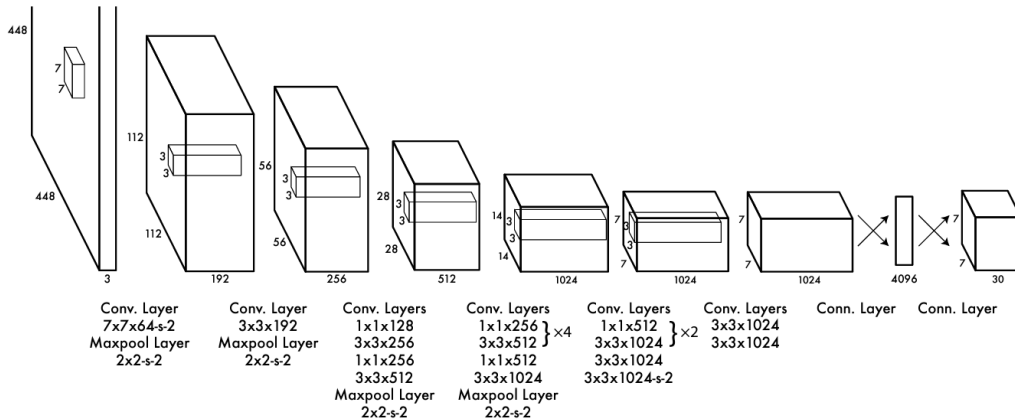


Figure 3: The Architecture

The YOLO loss function is designed to optimize both the localization and classification accuracy of the model, composed of three main components:

1. **Localization Loss (Bounding Box Regression Loss):** Measures the error between the predicted bounding box coordinates and the ground truth coordinates.
2. **Confidence Loss:** Penalizes the confidence score for bounding boxes, both when an object is present (objectness score) and when it is absent (no objectness score).
3. **Classification Loss:** Measures the error in predicting the class probabilities for the detected objects.

The overall loss function for YOLO is given by the formula:

$$\begin{aligned}
& \lambda_{coord} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{obj} [(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2] \\
& + \lambda_{coord} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{obj} \left[(\sqrt{w_i} - \sqrt{\hat{w}_i})^2 + (\sqrt{h_i} - \sqrt{\hat{h}_i})^2 \right] \\
& + \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{obj} [C_i - \hat{C}_i]^2 \\
& + \lambda_{noobj} \sum_{i=0}^{S^2} \sum_{j=0}^B \mathbb{1}_{ij}^{noobj} [C_i - \hat{C}_i]^2 \\
& + \sum_{i=0}^{S^2} \mathbb{1}_i^{obj} \sum_{c \in \text{classes}} [p_i(c) - \hat{p}_i(c)]^2,
\end{aligned}$$

where $\mathbb{1}_i^{obj}$ denotes if object appears in cell i and $\mathbb{1}_{ij}^{obj}$ denotes that the j th bounding box predictor in cell i is ‘responsible’ for that prediction.

II Applications

1 YOLO-based Object Detection Application

The YOLO model is sourced from <https://github.com/meituan>. To develop a web application, a yolov6 class with inference functionality based on the inferer of yolov6 is built (my_yolov6.py), adapted from the implementation available at https://github.com/thangnch/MiAI_YOLOv6_Deploy. The inference phase will utilize the pre-trained weights from 'yolov6s.pt' and the data configuration specified in 'coco.yaml'. The web application is built using Flask.

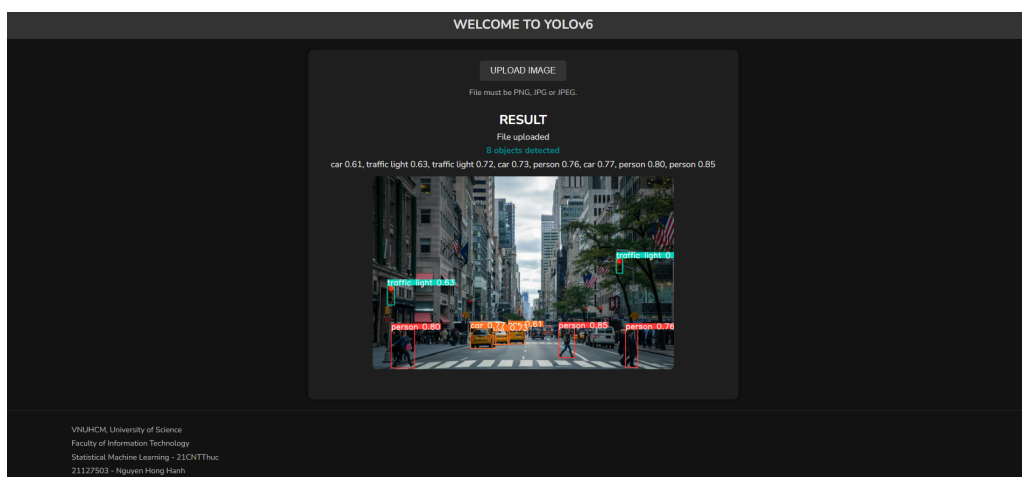


Figure 4: YOLO-based Object Detection Application

2 Vietnamese Food Detection Application

The dataset consists of 1000 images categorized into 5 classes: Pho, Bun, Bot Chien, Banh Mi, Goi Cuon. These images are primarily sourced from reviews of restaurants on Google Maps, TikTok, and YouTube, then annotated and generated in YOLO's TXT format using Roboflow. Details can be found at the following link: <https://universe.roboflow.com/nhh/vietnamese-food>. The sizes of the train, validation, and test sets are split according to a 70:20:10 ratio.

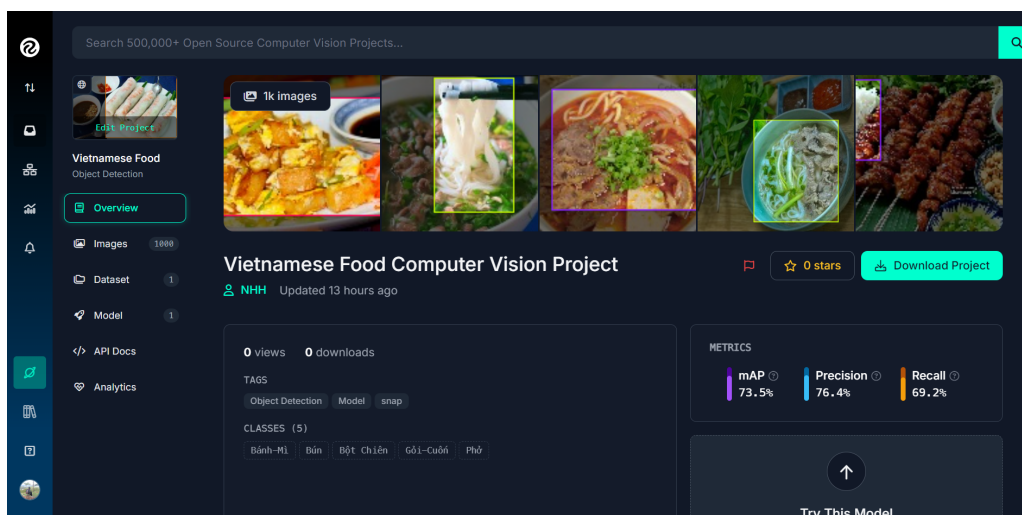


Figure 5: Dataset on Roboflow

The data is then added to the model to train a new model detecting Vietnamese Food.

The model was run for 158 out of 250 epochs (interrupted and stopped due to running out of GPU resources on Colab) with a batch size of 36. The result of mAP when evaluating the model after 20 epochs is plotted in figure 7.

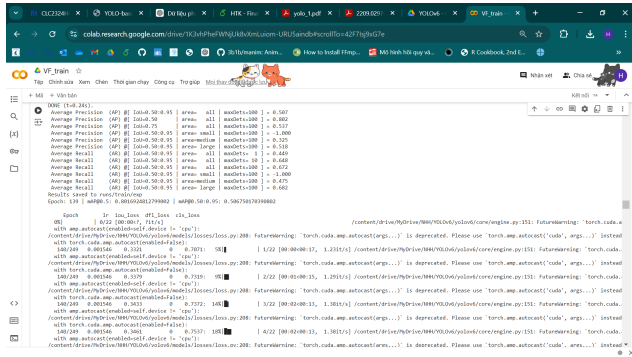


Figure 6: Training model with new objects' data

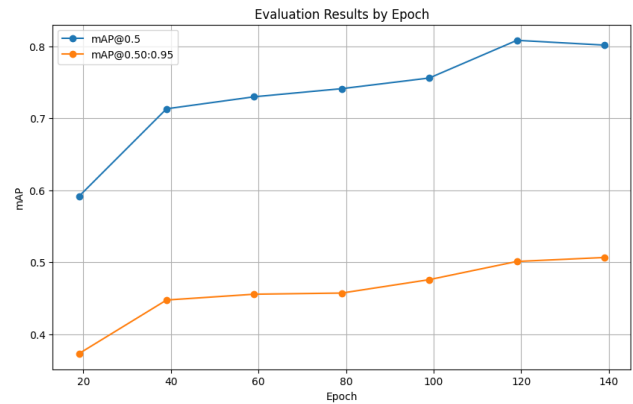


Figure 7: Evaluations Results by Epoch

The evaluation results on the validation and test sets are presented through confusion matrices, F1 curves, precision (P) curves, recall (R) curves, and precision-recall (PR) curves.

The results indicate that the model performs quite well at recognizing 'Banh Mi' and 'Bot Chien', while there is considerable confusion between 'Pho' and 'Bun'.

The results of running the web application with weights from the best checkpoint of the model trained on the Vietnamese food dataset are shown in figure 10.

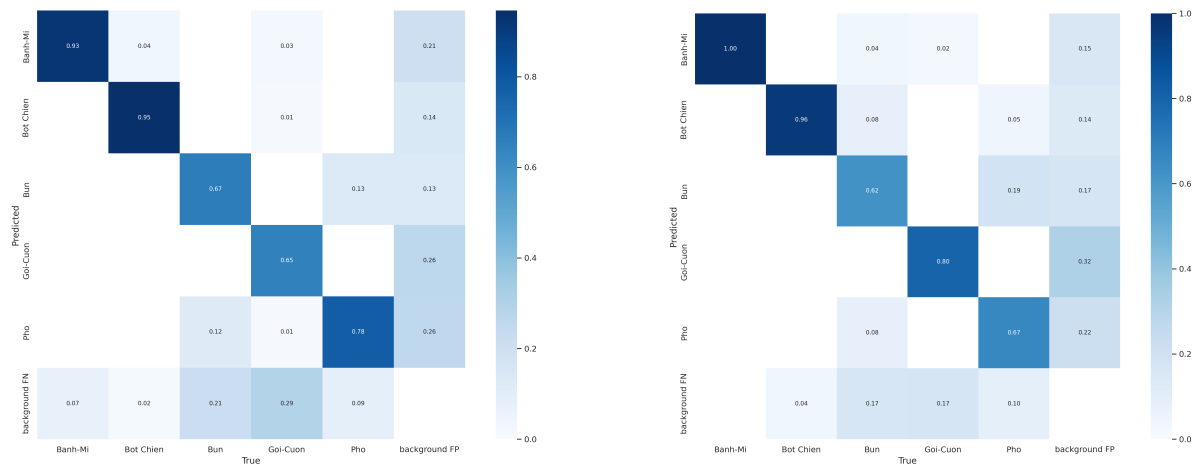


Figure 8: Confusion matrix of validation set (left) and test set (right)

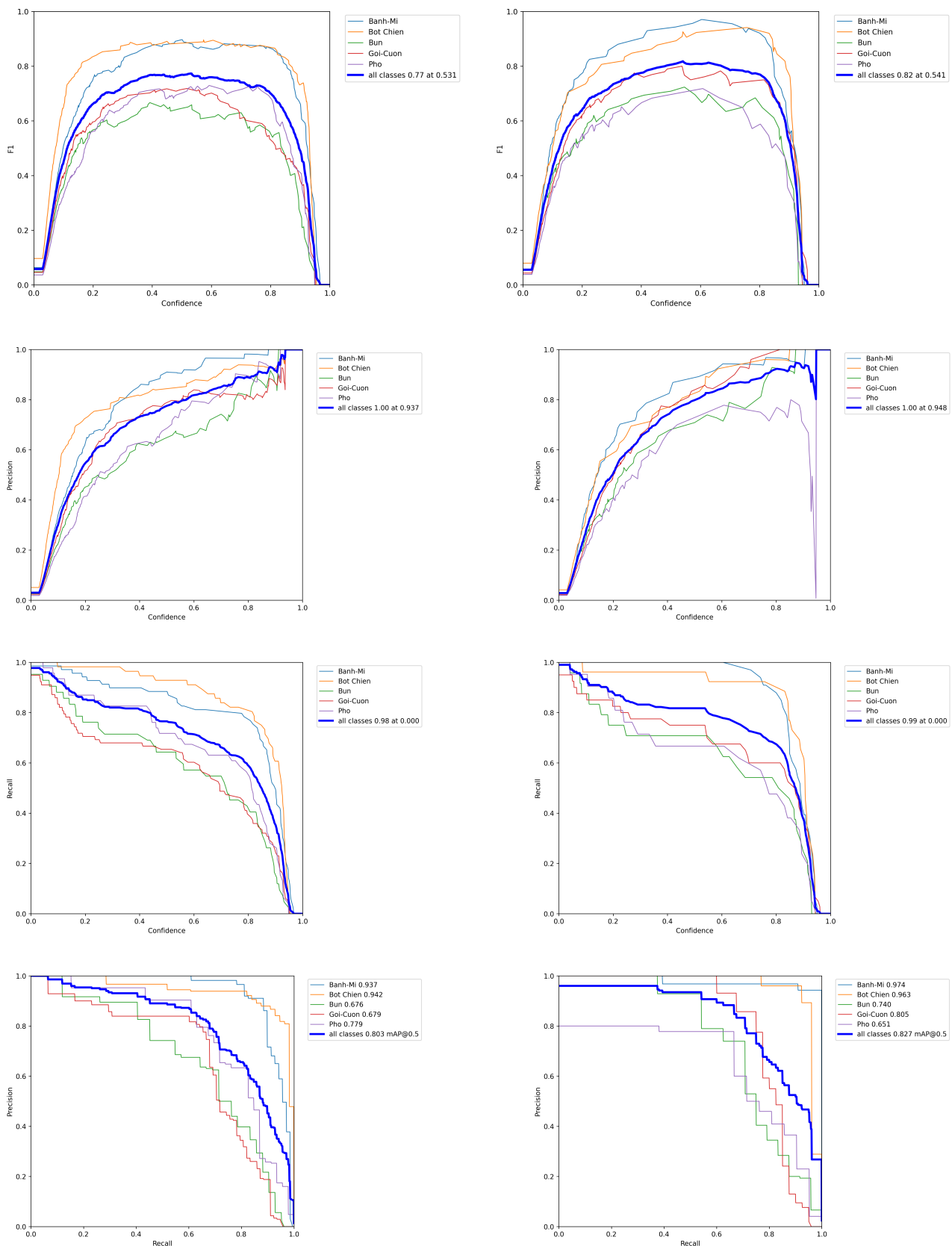


Figure 9: F1 curves, precision (P) curves, recall (R) curves, and precision-recall (PR) curves of validation set (left) and test set (right)

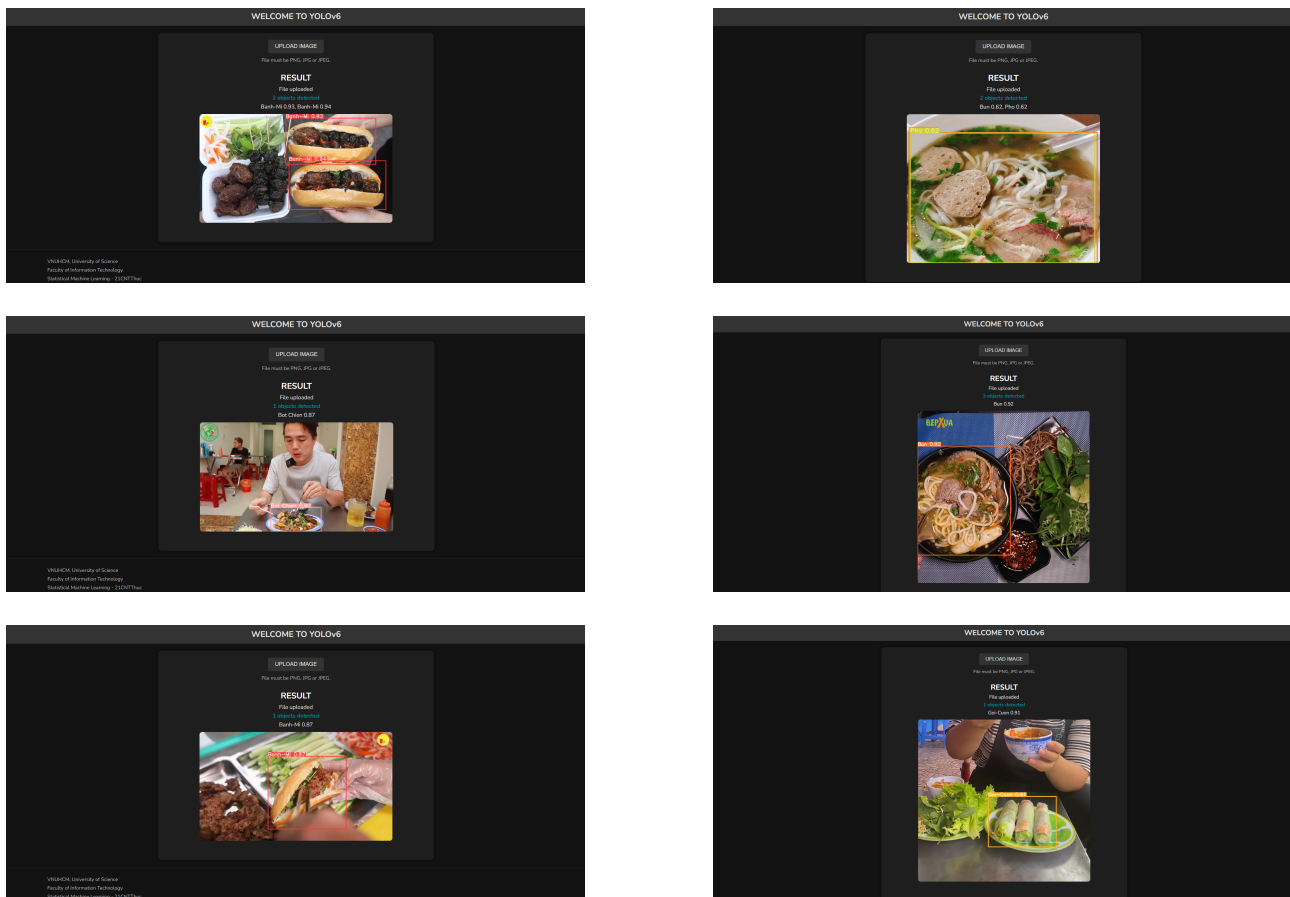


Figure 10: Results of running the web application with weights from the best checkpoint of the model trained on the Vietnamese food dataset

References

- Li, C., Li, L., Jiang, H., Weng, K., Geng, Y., Li, L., Ke, Z., Li, Q., Cheng, M., Nie, W., et al., 2022. Yolov6: A single-stage object detection framework for industrial applications. arXiv preprint arXiv:2209.02976 .
- Meituan, 2022. Yolov6: A single-stage object detection framework for industrial applications. <https://github.com/meituan/YOLOv6>.
- MIAI, . Yolov6 deployment. https://github.com/thangnch/MiAI_YOLOv6_Deploy.
- Redmon, J., Divvala, S., Girshick, R., Farhadi, A., 2016. You only look once: Unified, real-time object detection, in: Proceedings of the IEEE conference on computer vision and pattern recognition, pp. 779–788.